

추론 파이프라인: proposal_planning → CodeAgent

→ 최종 답변

작성일: 2026-04-25

대상 코드: `examples/open_deep_research/`

사용 언어: Python 3 / smolagents / sentence-transformers

1. 전체 흐름 개요

아래 흐름도는 GAIA 태스크 하나를 처리하는 전체 파이프라인입니다.

```
run_gaia.py → answer_single_question()
|
├─ [입력] example = { question, file_name, true_answer, ... }
|
├─ [KB 조회] AKBClient → odysseuss_db.jsonl
|   retrieval_type: hybrid / text / semantic / type_and_domain
|
├─ [proposal_planning()] planner_kb/planner.py
|   Stage 1a generate_knowledge → knowledge_draft
|   Stage 1b refine_knowledge → knowledge_text (가이드 있을 때 만)
|   Stage 2a generate_plan → plan_draft
|   Stage 2b refine_plan → plan_str (가이드 있을 때 만)
|   Stage 3a generate_instance → instance_draft
|   Stage 3b refine_instance → instance_str (가이드 있을 때 만)
|
├─ [CodeAgent.run()] smolagents
|   additional_knowledge = "Knowledge:\n...\nPlan:\n...\nInstance:\n..."
|   PlanningStep + ReAct Loop (Thought → Code → Observation)
|
└─ [prepare_response()] scripts/reformulator.py
    agent 메모리 → 단답형 최종 답변
```

주요 모듈

모듈	역할
<code>run_gaia.py</code>	전체 오케스트레이터, <code>answer_single_question()</code> 진입점
<code>agent_kb/</code> <code>agent_kb_retrieval.py</code>	KB 인덱싱 및 검색 (<code>AKB_Manager</code>)

모듈	역할
<code>planner_kb/planner.py</code>	<code>proposal_planning()</code> — 7단계 생성·교정
<code>planner_kb/ planner_prompts.yaml</code>	프롬프트 템플릿
<code>smolagents.CodeAgent</code>	ReAct 루프 실행
<code>scripts/reformulator.py</code>	최종 답변 추출

2. KB 조회 단계

2.1 데이터 소스 — `odysseuss_db.jsonl`

각 레코드는 과거 태스크 경험 하나를 표현합니다.

필드	타입	설명
<code>task_id</code>	str	레코드 고유 ID
<code>task</code>	str	태스크 질문 텍스트
<code>true_answer</code>	str	정답 레이블
<code>task_analysis</code>	dict	{ knowledge, plan, task_type, domain }
<code>agent_planning</code>	str/null	에이전트가 생성한 원시 계획
<code>decision_augmentation</code>	dict	{ final_reference, signals_summary }

로딩 시 정규화 처리:

```
# task_type / domain : dict → list 변환
if isinstance(ta_type, dict):
    task_analysis["task_type"] = [ta_type.get("raw"), ta_type.get("normalized")]

# final_reference 의 knowledge/plan 주입
if not task_analysis.get("knowledge") and fr.get("knowledge"):
    task_analysis["knowledge"] = fr["knowledge"]
```

2.2 검색 모드

hybrid_search — TF-IDF와 semantic 점수의 가중 합산

```
score = weight["text"] × TF-IDF cosine similarity
       + weight["semantic"] × all-MiniLM-L6-v2 cosine similarity
기본 가중치: { "text": 0.5, "semantic": 0.5 }
```

search_by_text — TF-IDF 벡터라이저로 task 필드 검색

search_by_semantic — sentence-transformers 임베딩 후 cosine 검색

type_domain_text_search — 유형·도메인 오버랩 + TF-IDF 가중 합산

```
final_score = 0.3 × type_score + 0.3 × domain_score + 0.4 × text_score
type_score   = |query_types ∩ kb_types| / max(|query_types|, 1)
domain_score = |query_domains ∩ kb_domains| / max(|query_domains|, 1)
```

2.3 검색 결과 스키마

```
{
  "task_id":          str,
  "total_score":       float,
  "task":             str,
  "true_answer":       str,
  "task_analysis": {
    "knowledge": str,
    "plan":      list[str],
    "task_type": list[str],
    "domain":    list[str],
  },
  "instance":         str | None, #
  decision_augmentation.final_reference.instance
  "decision_guide":   list | None, #
  decision_augmentation.signals_summary
}
```

Decision Guide 항목 구조:

```
{
  "level":          "knowledge" | "plan" | "instance",
  "failures":       list[str],
  "causes":         list[str],
  "corrections":    list[str],
}
```

3. proposal_planning 함수

3.1 함수 시그니처

```
def proposal_planning(
    example,                                # GAIA 태스크 dict
    augmented_question: str,                # 파일 설명 포함 확장 질문
    model_name: str,
    key: str,
    url: str,
    model: Any,
    slm: bool,
    retrieval_method: Callable,
    top_k: int,
    retrieval_option: str = "task_text",    # "task_text" |
    "type_and_domain"
    type_domain_retrieval_method = None,
    plan_mode: str | None = None,
    tools = None,
```

```

    managed_agents = None,
    planning_prompt_templates = None,
) -> dict

```

반환값:

```

{
    "plan": str, # CodeAgent에 주입되는 최종 컨텍스트
    "knowledge": str, # Stage 1b 출력 (refined knowledge)
    "plan_steps": str, # Stage 2b 출력 (refined plan)
    "instance": str, # Stage 3b 출력 (refined instance)
    "retrieval_results": list,
    "examples": dict,
}

```

3.2 단계 [1] — 유사 태스크 검색

현재 질문과 가장 유사한 KB 레코드 top_k개를 가져와 few-shot 예시와 Decision Guide로 활용합니다.

```

# retrieval_option == "task_text" (기본)
retrieval_results = retrieval_method(example["question"], top_k=top_k)

# retrieval_option == "type_and_domain"
classification =
classify_task_type_and_domain(task=augmented_question, ...)
retrieval_results = type_domain_retrieval_method(
    example["question"],
    task_types=classification["task_type_normalized"],
    domains=classification["domain_normalized"],
    top_k=top_k,
)

```

검색 후 6개의 예시·가이드 블록을 구성합니다:

```

knowledge_examples = build_similar_task_direction_blocks(retrieval_results)
plan_examples = build_similar_task_plan_blocks(retrieval_results)
instance_examples = build_instance_examples(retrieval_results)
guide_knowledge = build_decision_guide_blocks(retrieval_results, level="knowledge")
guide_plan = build_decision_guide_blocks(retrieval_results, level="plan")
guide_instance = build_decision_guide_blocks(retrieval_results, level="instance")

```

3.3 Stage 1a — generate_knowledge

목적: 태스크를 풀기 위해 필요한 도메인 지식(선언적 + 절차적)을 생성합니다.

함수 시그니처:

```
def generate_knowledge(
    task: str,
    examples: str,          # knowledge_examples 블록
    planning_prompt_template: dict,
    model_name, key, url, model, slm,
) -> str
```

프롬프트 (`knowledge_prompt`):

```
Extract the domain knowledge required to understand and solve the given
task.
Provide both:
    (a) declarative knowledge: domain concepts and definitions used
    (b) procedural knowledge: methodology, paradigm, or algorithm to
    apply
Return only the knowledge as plain text. Do NOT include the direct
answer.

TASK:
{{task}}

SIMILAR TASKS:
{{examples}}
```

출력: `knowledge_draft` — 선언적·절차적 지식 텍스트

3.4 Stage 1b — refine_knowledge (Decision Guide 교정)

목적: Stage 1a 초안을 Decision Guide 신호로 교정합니다. Decision Guide가 없으면 초안을 그대로 반환합니다.

```
def refine_knowledge(task, draft, decision_guide, ...) -> str:
    if not decision_guide:
        return draft      # 가이드 없으면 스킵
    ...
```

프롬프트 (`refine_knowledge_prompt`):

```
You generated domain knowledge for a task. A Decision Guide identifies f
ailure
patterns observed in similar tasks.
Refine the generated knowledge to address the failures. Only modify
what the
guide says is insufficient or wrong.
Preserve all correct parts. Return the full refined knowledge as plain t
ext.

TASK: {{task}}
GENERATED KNOWLEDGE: {{draft}}
DECISION GUIDE: {{decision_guide}}
```

출력: `knowledge_text` — 교정된 최종 지식

3.5 Stage 2a — generate_plan

목적: 지식과 유사 예시를 바탕으로 최대 10단계 고수준 계획을 생성합니다.

프롬프트 (`plan_prompt`):

```
Develop a high-level plan of no more than 10 steps to solve the given task
based only on the provided knowledge and similar tasks.
The plan must not include the final answer and the final conclusion.

Requirements:
- Use the knowledge explicitly and progressively in each step.
- Derive a decision criterion: an intermediate, task-specific quantity,
  rule, or structure that can be used to select the answer.

Strict Output Format:
- <step>
- <step>
- ...

TASK: {{task}}
KNOWLEDGE: {{knowledge}}
SIMILAR TASKS: {{examples}}
```

출력: `plan_draft` — bullet-list 형식 계획

3.6 Stage 2b — refine_plan (Decision Guide 교정)

프롬프트 (`refine_plan_prompt`):

```
You generated a solution plan for a task. A Decision Guide identifies f
ailure
patterns observed in similar tasks.
Refine the generated plan to address the failures. Only modify steps th
at the
guide says are wrong or missing.
Preserve all correct steps. Return the full refined plan as a bullet li
st.

TASK: {{task}}
KNOWLEDGE: {{knowledge}}
GENERATED PLAN: {{draft}}
DECISION GUIDE: {{decision_guide}}
```

출력: `plan_str` — 교정된 최종 계획

3.7 Stage 3a — generate_instance

목적: 계획을 실제 태스크에 구체적으로 적용한 실행 인스턴스를 생성합니다. 중간 계산값과 추론 과정을 포함하되 최종 답은 제외합니다.

프롬프트 (`instance_prompt`):

Generate a concrete execution instance that grounds the given plan into specific, actionable steps with actual values and reasoning. The instance should demonstrate exactly how to apply the plan to THIS task, showing intermediate computations and logical deductions.

Requirements:

- Walk through each plan step with concrete values from the task.
- Show actual computations, lookups, or reasoning steps.
- Do NOT reveal or guess the final answer – stop just before the conclusion.

TASK: {{task}}
KNOWLEDGE: {{knowledge}}
PLAN: {{plan}}
SIMILAR INSTANCES: {{examples}}

출력: `instance_draft` — 구체적 실행 인스턴스 텍스트

3.8 Stage 3b — refine_instance (Decision Guide 교정)

프롬프트 (`refine_instance_prompt`):

You generated a concrete execution instance for a task. A Decision Guide identifies failure patterns observed in similar tasks. Refine the instance to fix the failures identified in the guide. Only modify what is incorrect or incomplete. Preserve all correct parts. Return the full refined execution instance.

TASK: {{task}}
KNOWLEDGE: {{knowledge}}
PLAN: {{plan}}
GENERATED INSTANCE: {{draft}}
DECISION GUIDE: {{decision_guide}}

출력: `instance_str` — 교정된 최종 인스턴스

3.9 최종 plan 문자열 조합

```
# plan_mode == None 또는 "plan" (기본)
final_plan = (
    f"Knowledge:\n{knowledge_text}\n\n"
    f"Plan:\n{plan_str}\n\n"
    f"Instance:\n{instance_str}"
)
```

4. Generate-then-Refine 전략

4.1 흐름도

```
KB 유사 태스크 검색
|
├─ build_decision_guide_blocks(level="knowledge") →
guide_knowledge
├─ build_decision_guide_blocks(level="plan") → guide_plan
└─ build_decision_guide_blocks(level="instance") →
guide_instance

Stage 1a: generate_knowledge(task, examples) → knowledge_draft
Stage 1b: refine_knowledge(draft, guide_knowledge) → knowledge_text
                                                (가이드 없으면 draft
반환)

Stage 2a: generate_plan(task, knowledge, examples) → plan_draft
Stage 2b: refine_plan(draft, guide_plan) → plan_str
                                                (가이드 없으면 draft
반환)

Stage 3a: generate_instance(task, knowledge, plan, examples) →
instance_draft
Stage 3b: refine_instance(draft, guide_instance) →
instance_str
                                                (가이드 없으
면 draft 반환)
```

4.2 Decision Guide 신호 구조

```
# odysseuss_db.jsonl: decision_augmentation.signals_summary
[
  {
    "level":      "knowledge" | "plan" | "instance",
    "failures":   ["실패 현상 설명 ..."],
    "causes":     ["근본 원인 ..."],
    "corrections": ["수정 방법 ..."],
  },
  ...
]
```

`build_decision_guide_blocks()` 가 이를 프롬프트용 텍스트로 변환합니다:

```
[From Task: <task text>]
[KNOWLEDGE] Failure: ...
              Cause: ...
              Correction: ...
```


4.3 Generate-then-Refine 방식을 선택한 이유

전략	특성
가이드와 함께 한 번에 생성	SLM은 긴 가이드 조건이 프롬프트 앞에 오면 컨텍스트 부담으로 초안 품질 저하
Generate-then-Refine	제약 없이 최선의 초안 생성 → 짧고 집중된 교정 프롬프트로 수정

각 refine 함수는 `if not decision_guide: return draft` 가드로, KB에 해당 레벨 가이드가 없으면 불필요한 LLM 호출을 방지합니다.

5. CodeAgent 실행

5.1 에이전트 생성

```
manager_agent = CodeAgent(  
    model=model, # TogetherAI SLM 또는  
    OpenAI 모델  
    tools=[],  
    max_steps=args.max_steps, # 기본 12  
    planning_interval=args.planning_interval, # 기본 1 (매 스텝마다 재  
    계획)  
    additional_authorized_imports=AUTHORIZED_IMPORTS,  
    agent_kb=args.agent_kb,  
    top_k=args.top_k,  
    retrieval_type=args.retrieval_type,  
    plan_mode=args.plan_mode,  
)
```

`AUTHORIZED_IMPORTS`: `requests`, `pandas`, `numpy`, `sympy`, `sklearn`, `scipy`, `PIL`, `PyPDF2`, `torch` 등 25개 라이브러리

5.2 additional_knowledge 주입

`proposal_planning()`의 반환값 `plan["plan"]`이 `additional_knowledge`로 에이전트에 전달됩니다.

```
# run_gaia.py  
if args.kb_type == "proposal":  
    def planner_fn(task, tools, managed_agents) -> dict:  
        return proposal_planning(  
            example=example,  
            augmented_question=augmented_question,  
            plan_mode=args.plan_mode,  
            ...  
        )  
    agent.planner_fn = planner_fn  
  
final_result = agent.run(  
    augmented_question,
```

```

        additional_knowledge=_additional_knowledge_for_planning
    )

```

에이전트 시스템 프롬프트에 삽입되는 내용:

```

Knowledge:
<refined knowledge text>

Plan:
<refined plan bullet list>

Instance:
<refined execution instance>

```

5.3 ReAct 루프

```

Iteration 1 ~ max_steps:
┌ PlanningStep (planning_interval마다) ───────────────────────────────────┐
│ LLM이 현재 상태를 보고 high-level plan 재작성 │
└──────────────────────────────────────────────────────────────────────────┘

┌ ActionStep ─────────────────────────────────────────────────────────────────┐
│ Thought      : LLM이 다음 행동을 Python 코드로 작성 │
│ Code         : 에이전트가 Python 코드 실행 │
│ Observation  : 코드 실행 결과 (stdout/반환값) 수집 │
└──────────────────────────────────────────────────────────────────────────┘

max_steps 초과 또는 final_answer() 호출 시 종료

```

5.4 사용 가능한 도구

도구	클래스	역할
SearchTool	<code>scripts/searcher.py</code>	Tavily / Exa 웹 검색
CrawlerReadTool	<code>scripts/async_web_crawler.py</code>	URL 본문 크롤링
CrawlerArchiveSearchTool	<code>scripts/async_web_crawler.py</code>	Wayback Machine 검색
TextInspectorTool	<code>scripts/text_inspector_tool.py</code>	PDF/텍스트 파일 검사
AudioInspectorTool	<code>scripts/audio_inspector_tool.py</code>	오디오 파일 검사
VisualInspectorTool	<code>scripts/visual_inspector_tool.py</code>	이미지 파일 검사

5.5 메모리 타입

타입	용도
TaskStep	최초 태스크와 additional_knowledge
PlanningStep	매 planning_interval마다 LLM이 작성하는 고수준 계획
ActionStep	(Thought, Code, Observation) 삼중 쌍

6. 최종 답변 추출

6.1 prepare_response() — scripts/reformulator.py

```
def prepare_response(  
    original_task: str,  
    inner_messages,          # agent.write_memory_to_messages() 결과  
    reformulation_model: Model,  
    multiple: bool = False,  
    ) -> str
```

처리 과정:

1. 시스템 메시지에 원래 태스크 제시
2. 에이전트 메모리(`inner_messages`)를 USER 역할로 변환
3. 최종 지시 메시지 추가 — 단답형 포맷 규칙 명시
4. `reformulation_model(messages).content` 호출
5. `response.split("FINAL ANSWER: ")[-1].strip()` 로 최종 답 추출

최종 지시 메시지의 포맷 규칙:

- 숫자: 숫자만 사용, 쉼표·단위 제거 (단, 질문에서 요구하면 유지)
- 텍스트: 관사·약어 제거, 마침표 제외
- 목록: 쉼표로 구분
- 답을 알 수 없으면: "Unable to determine"

출력: 정제된 단답형 최종 답변 문자열

7. 데이터 스키마

7.1 odyseuss_db.jsonl 레코드

```
{  
  "task_id": "string",  
  "task": "string",  
  "true_answer": "string",  
  "agent_planning": "string | null",  
  "task_analysis": {  
    "knowledge": "string",
```

```

    "plan": ["step1", "step2", "..."],
    "task_type": ["computation", "analysis", "..."],
    "domain": ["mathematics", "science", "..."]
  },
  "decision_augmentation": {
    "final_reference": {
      "knowledge": "string",
      "plan": ["step1", "..."],
      "instance": "string"
    },
    "signals_summary": [
      {
        "level": "knowledge | plan | instance",
        "failures": ["string"],
        "causes": ["string"],
        "corrections": ["string"]
      }
    ]
  }
}

```

7.2 proposal_planning() 반환값

```

{
  "plan": str,
  # "Knowledge:\n{knowledge_text}\n\nPlan:\n{plan_str}\n\nInstance:\n{instance_str}"

  "knowledge": str, # Stage 1b 출력
  "plan_steps": str, # Stage 2b 출력
  "instance": str, # Stage 3b 출력
  "retrieval_results": list, # KB 검색 결과 원본
  "examples": dict,
}

```

부록. 모듈 파일 경로

역할	파일 경로
전체 실행 진입점	examples/open_deep_research/run_gaia.py
KB 인덱싱·검색	examples/open_deep_research/agent_kb/ agent_kb_retrieval.py
KB 서비스	examples/open_deep_research/agent_kb/ agent_kb_service.py
Proposal Planning	examples/open_deep_research/planner_kb/planner.py
프롬프트 템플릿	examples/open_deep_research/planner_kb/ planner_prompts.yaml
Planner 공개 API	examples/open_deep_research/planner_kb/__init__.py

역할	파일 경로
최종 답변 추출	<code>examples/open_deep_research/scripts/reformulator.py</code>
모델 헬퍼	<code>examples/open_deep_research/scripts/automodel.py</code>